



Universidad Autónoma de Nuevo León
Facultad de Ciencias Físico Matemáticas

Laboratorio de Programación Orientada a Objetos

Avance — Laboratorio 03: Encapsulamiento y Modificadores de Acceso

Alumno: Daniel Antonio Chávez Peña

Matrícula: 1956765

Materia: Laboratorio de Programación Orientada a Objetos

Descripción de la práctica

Esta práctica tiene como objetivo aplicar el principio de encapsulamiento en Java mediante el uso de modificadores de acceso (`public`, `private` y `protected`), así como la implementación de métodos `getter` y `setter` con validaciones básicas para proteger el estado interno de los objetos.

El escenario propuesto consiste en modelar un sistema de transporte multimodal para una empresa de logística, el cual gestiona distintos tipos de vehículos: automóviles, aviones y barcos. Para ello se diseñó una clase base **Vehiculo**, que concentra los atributos y comportamientos comunes a todos los vehículos (marca, modelo, año y velocidad máxima), aplicando `private` a los datos que no deben exponerse directamente y `protected` al atributo `velocidadMax`, de forma que las subclases puedan acceder a él sin necesidad de un `getter`.

A partir de esta clase base se generaron tres subclases mediante herencia: **Automovil**, **Avion** y **Barco**, cada una con sus propios atributos particulares (número de puertas y tipo de motor, número de motores y altitud máxima, tipo de casco y tonelaje, respectivamente). En cada subclase se incorporaron `setters` con validación, de manera que valores fuera de rango (como un año no válido o un número de puertas incorrecto) generen un mensaje de error en lugar de modificar el estado del objeto.

Finalmente, en la clase **Main** se instanció un objeto de cada tipo de vehículo, se probaron las validaciones de los `setters` con datos incorrectos a propósito y posteriormente se corrigieron con valores válidos, confirmando el correcto funcionamiento del encapsulamiento implementado.

Evidencia de ejecución

A continuación se muestra la salida en consola obtenida al ejecutar la clase principal **Main**, donde se observa la creación de los objetos, la impresión de su información mediante `toString()` y los mensajes de error generados por las validaciones de los `setters`.

```
PS C:\Users\Ddani28\Documents\LabP00\Practica3> java Main.java
=== Sistema de Transporte Multimodal ===

-- Automovil --
Marca: Toyota | Modelo: Corolla | Año: 2022 | Vel. Máx: 180.0 km/h
Puertas: 4 | Eléctrico: No

Error: año no válido.
Error: número de puertas no válido.

Marca: Toyota | Modelo: Corolla | Año: 2023 | Vel. Máx: 180.0 km/h
Puertas: 4 | Eléctrico: No

-- Avion --
Marca: Boeing | Modelo: 737 | Año: 2019 | Vel. Máx: 850.0 km/h
Motores: 2 | Altitud Máx: 12500.0 m | Aerolínea: Aeroméxico

Error: altitud máxima no válida.

-- Barco --
Marca: Ferretti | Modelo: 550 | Año: 2020 | Vel. Máx: 45.0 km/h
Casco: Fibra de vidrio | Tonelaje Máx: 30.5 t | Tripulantes: 6
```

Conclusión

Esta práctica me ayudó a entender mejor por qué el encapsulamiento es tan útil: en vez de dejar que cualquier parte del código modifique los atributos directamente, los setters permiten filtrar valores que no tienen sentido (como un año inválido o una altitud negativa) antes de que rompan el estado del objeto. También quedó más claro el uso de `protected`, ya que a diferencia de `private`, permite que las subclases accedan al atributo sin pasar por un getter, lo cual fue cómodo al trabajar con `velocidadMax` en `Automovil`, `Avion` y `Barco`. En general fue un buen ejercicio para practicar herencia junto con buenas prácticas de acceso a los datos.